

Universidad de San Carlos de Guatemala

Facultad de Ingeniería

Escuela de Ciencias y Sistemas

Laboratorio de Sistemas Operativos 1

**Manual Técnico y Guía de Instalación**  
Proyecto 1

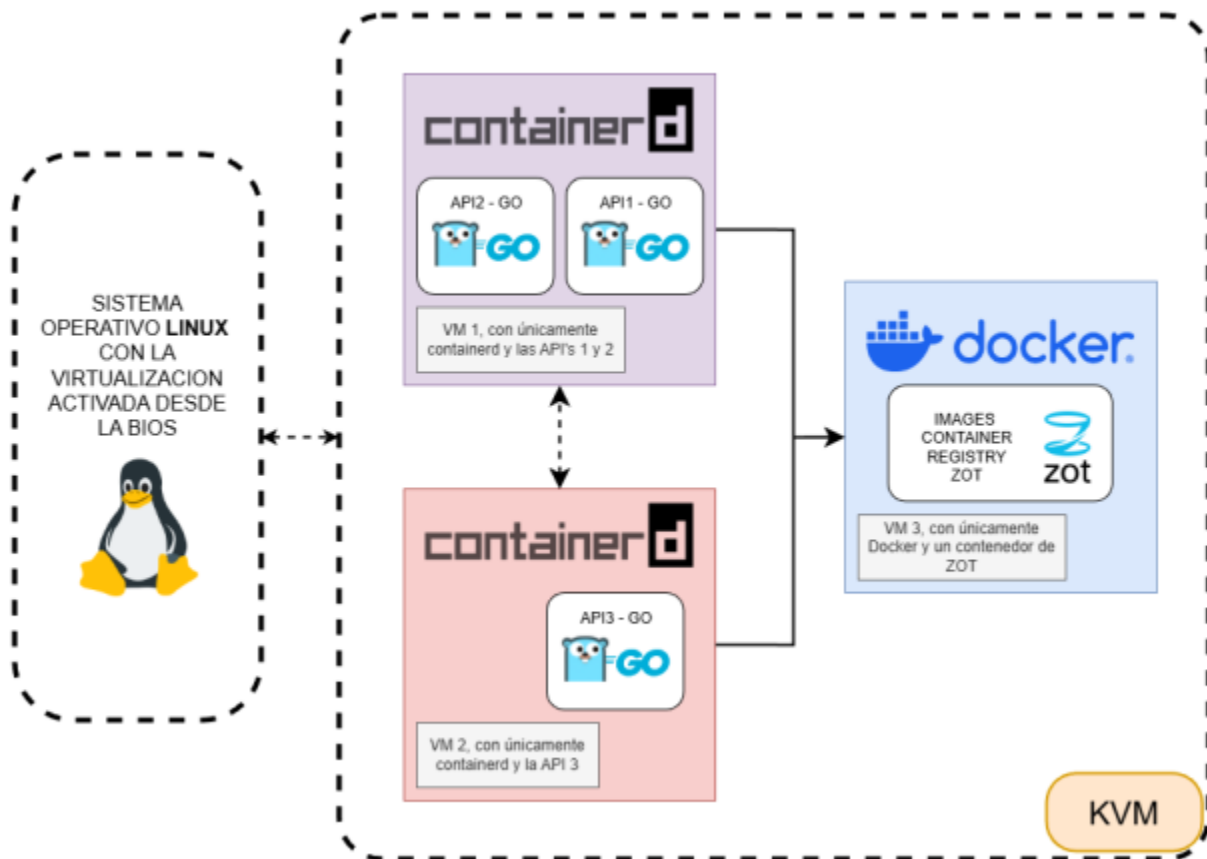
Sebastián Gómez Lavarreda  
201602929  
Guatemala, febrero de 2026

## Descripción General del Proyecto

El proyecto consiste en el desarrollo de 3 APIs las cuales serán contenerizadas y cargadas en 2 máquinas virtuales. Se utilizará un contenedor de Zot para almacenar las imágenes de las APIs en una tercera máquina virtual.

La virtualización se realiza por medio del programa Virt Manager, que utiliza qemu-kvm, creando máquinas virtuales del sistema operativo Ubuntu Server 22.04.5. Estas máquinas virtuales son agregadas a la red virtual creada por Virt Manager.

La VM1 aloja las API 1 y 2, utilizando Containerd para ejecutarlas. La VM2 aloja la API 3, utilizando también Containerd para ejecutarla. La VM3 utiliza Docker para ejecutar Zot. Se adjunta un diagrama de la distribución del sistema.



## Descripción de las API

Las APIs fueron desarrolladas en el lenguaje Go. Cada API obtiene las URL y puertos de las otras APIs por medio de variables de entorno configuradas en la VM que la aloja, para asegurar la seguridad del sistema.

Las API contienen 2 tipos de endpoints.

- Endpoint para obtener el estado de la API (health): Retorna un JSON que indica si la API está funcionando correctamente. El JSON contiene los campos:
  - status: "UP" si se encuentra corriendo la API.
  - message: "API# is ready" indicando el número de API que se llama.
  - timestamp: registro del tiempo actual de cuando se realiza la llamada.
  - VM: número de máquina virtual en la que se encuentra alojada la API.
  - carnet: 201602929 (el número de mi carnet).
- Endpoint para llamada a otra API: Retorna un JSON que indica si la API que se llamó está funcionando correctamente. El JSON contiene los campos:
  - apiname: "API#" indicando el nombre de la API llamada.
  - message: "The API# located on the VM# is working" indicando el nombre de la API llamada y el número de la VM donde está almacenada. En caso de no obtener una respuesta positiva de la llamada, este mensaje indica que la API no esta funcionando correctamente.
  - connection: "true" si la API llamada responde correctamente, "false" si no.
  - carnet: 201602929 (el número de mi carnet).

A continuación se describen los endpoints de cada API.

### API1

- GET /health: Indica el estado de la API1.
- GET /api1/201602929/call-api2: Llamada a la API2.
- GET /api1/201602929/call-api3: Llamada a la API3.

### API2

- GET /health: Indica el estado de la API2.
- GET /api2/201602929/call-api1: Llamada a la API1.
- GET /api2/201602929/call-api3: Llamada a la API3.

### API3

- GET /health: Indica el estado de la API3.
- GET /api3/201602929/call-api1: Llamada a la API1.
- GET /api3/201602929/call-api2: Llamada a la API2.

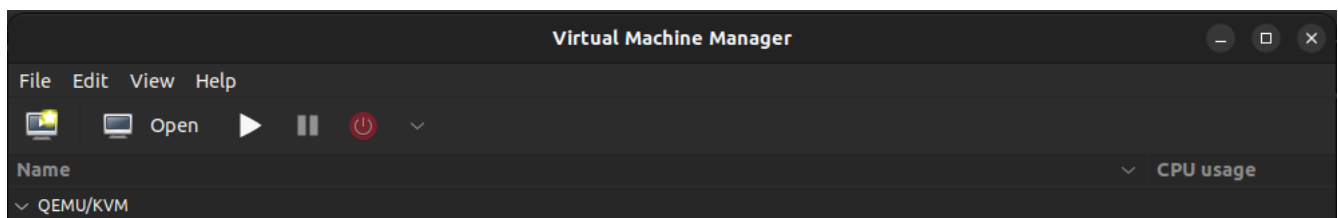
## Guía de Instalación

### Requisitos

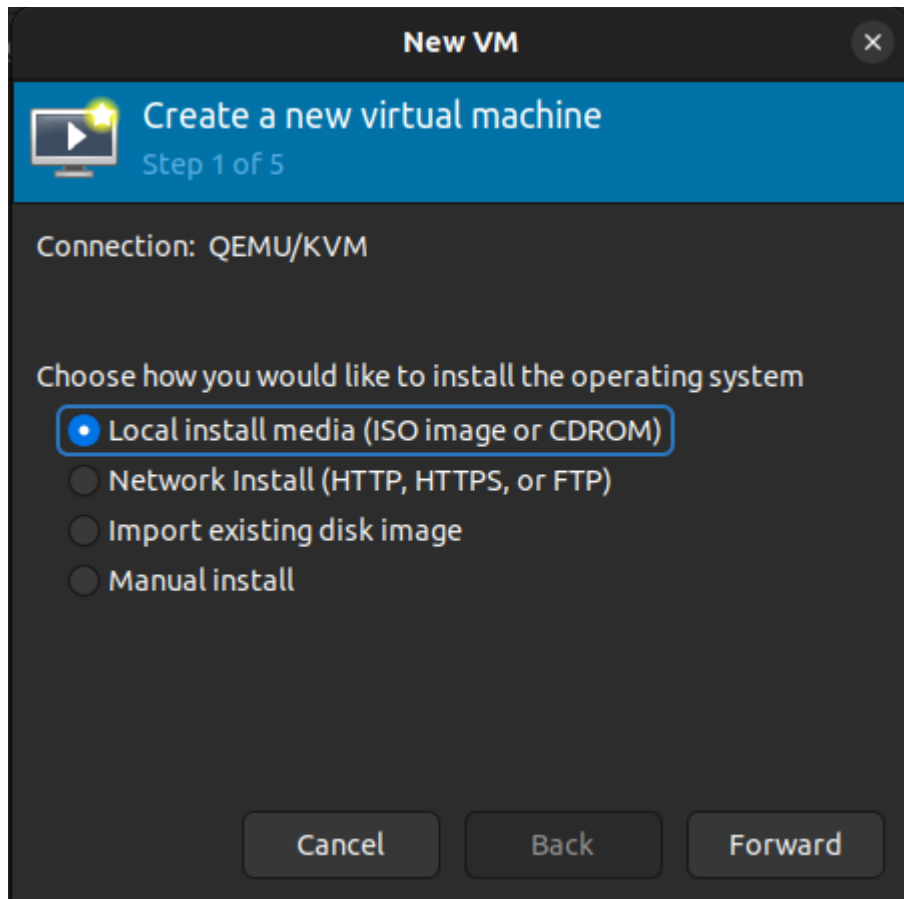
- Virt Manager
- ISO del sistema operativo para la instalación de las máquinas virtuales (Ubuntu Server 22.04.5 para esta guía).

### Pasos de Creación de VMs

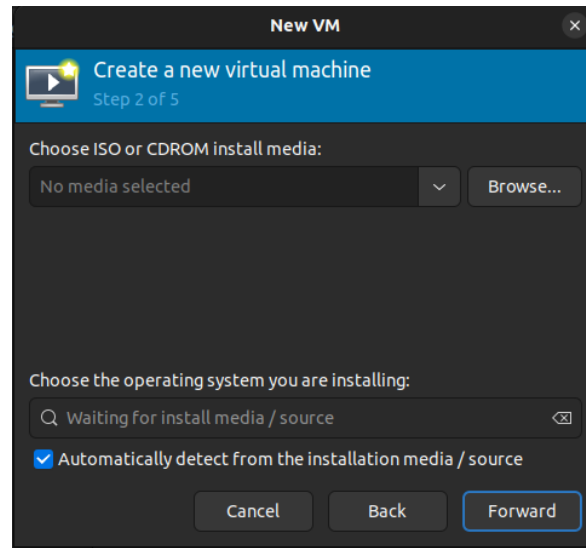
Desde Virt Manager, creamos una nueva máquina virtual presionando el botón de la pc con una estrella.



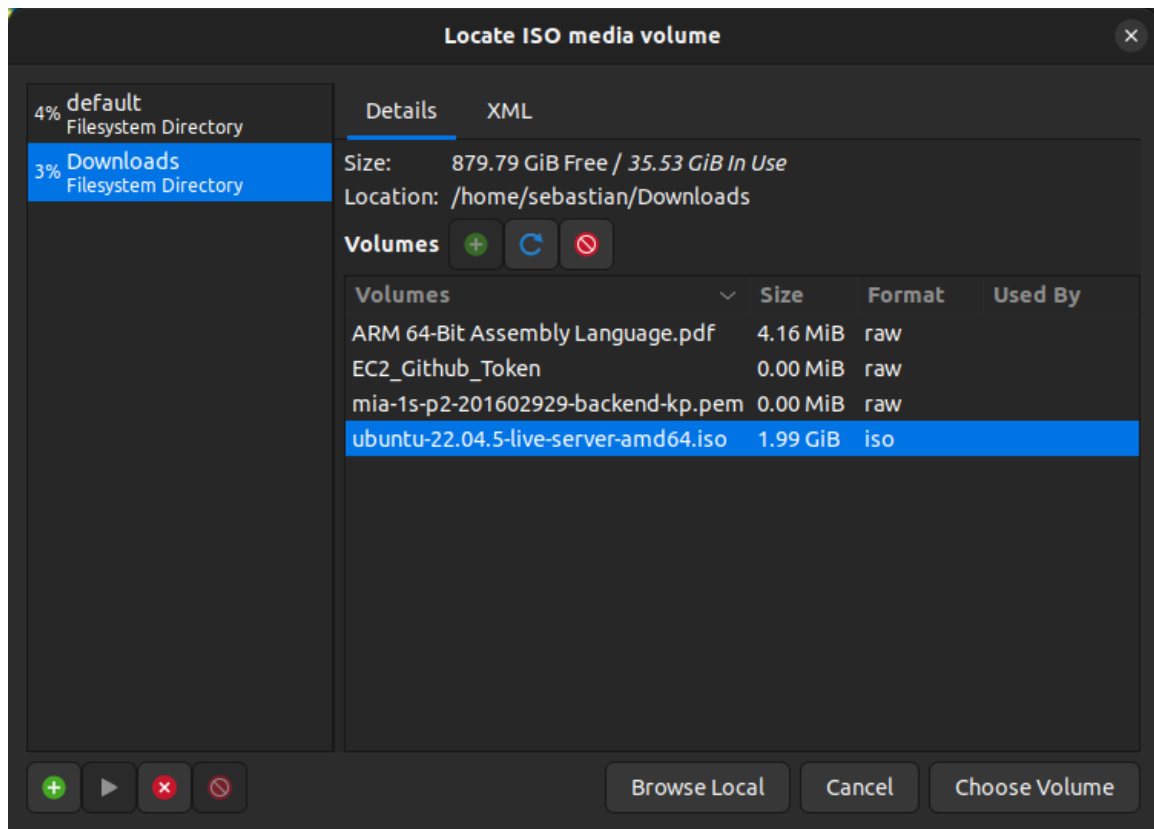
Seleccionamos la opción de “Medio de instalación local”. Damos clic en el botón “Forward”.



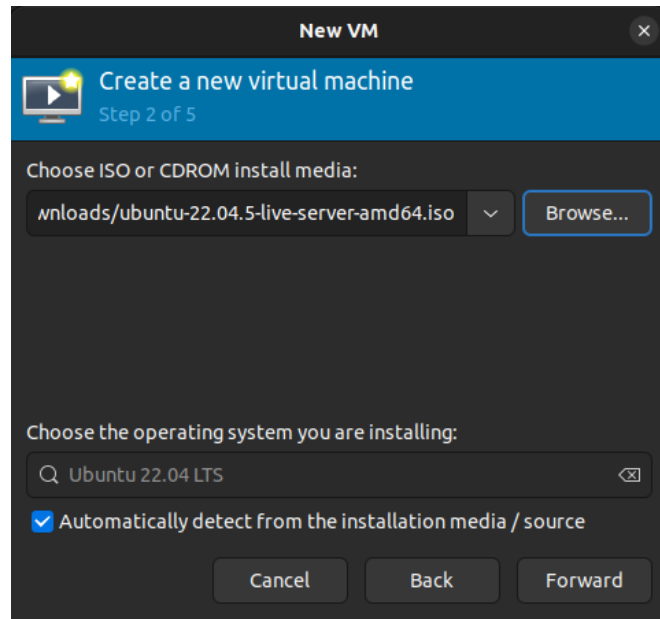
Damos clic en el botón “Browse” para seleccionar la ISO a utilizar.



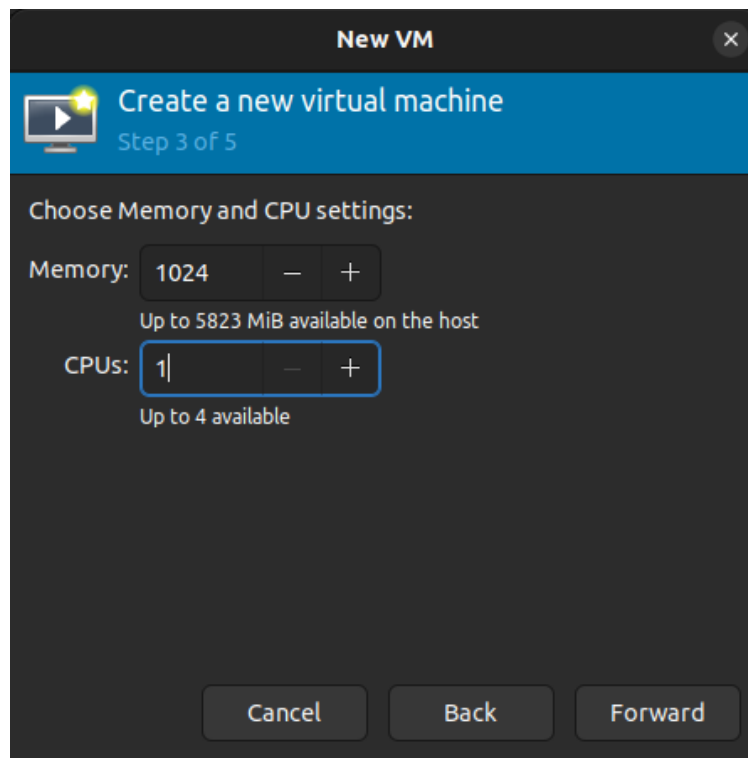
Seleccionamos la ISO y damos clic al botón “Choose Volume”.



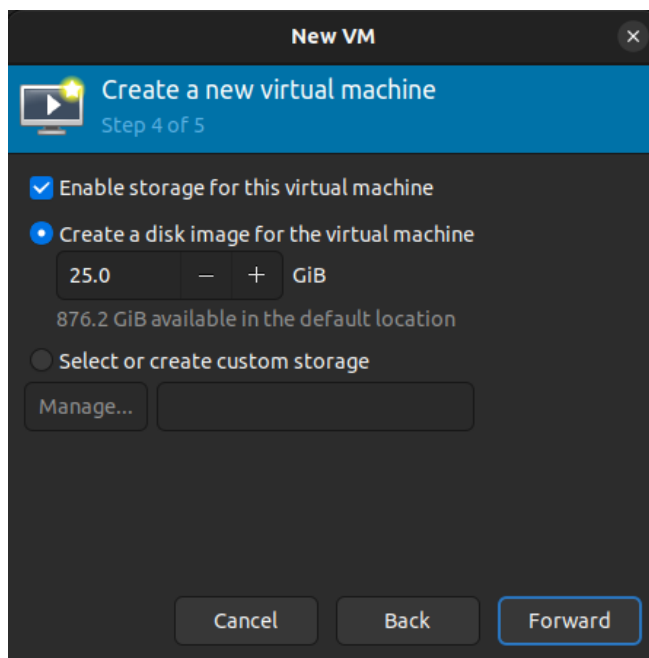
Damos clic al botón “Forward”.



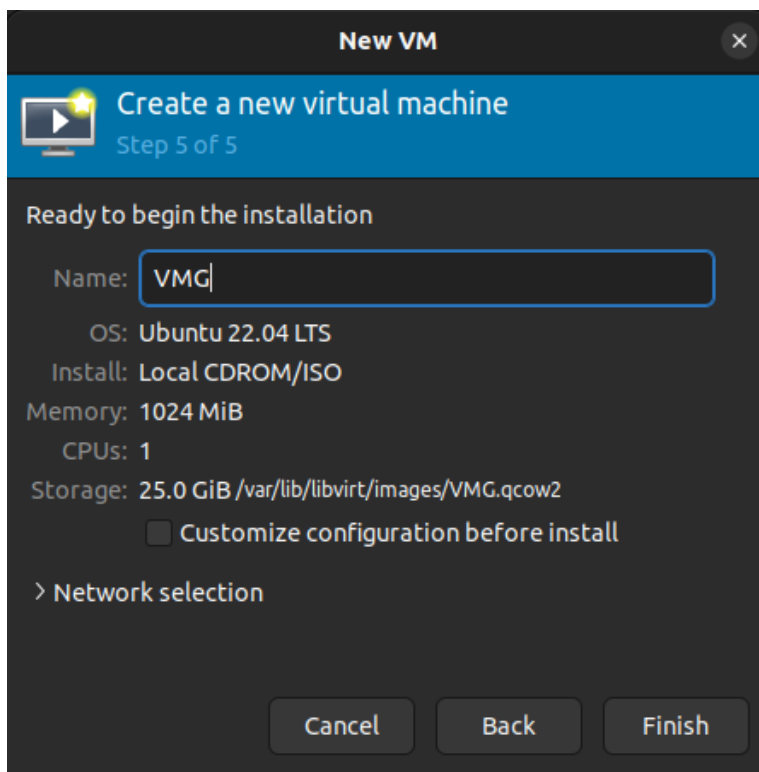
Ingresamos la cantidad de memoria que utilizaremos en la VM y la cantidad de nucleos que utilizará la VM de nuestro procesador.



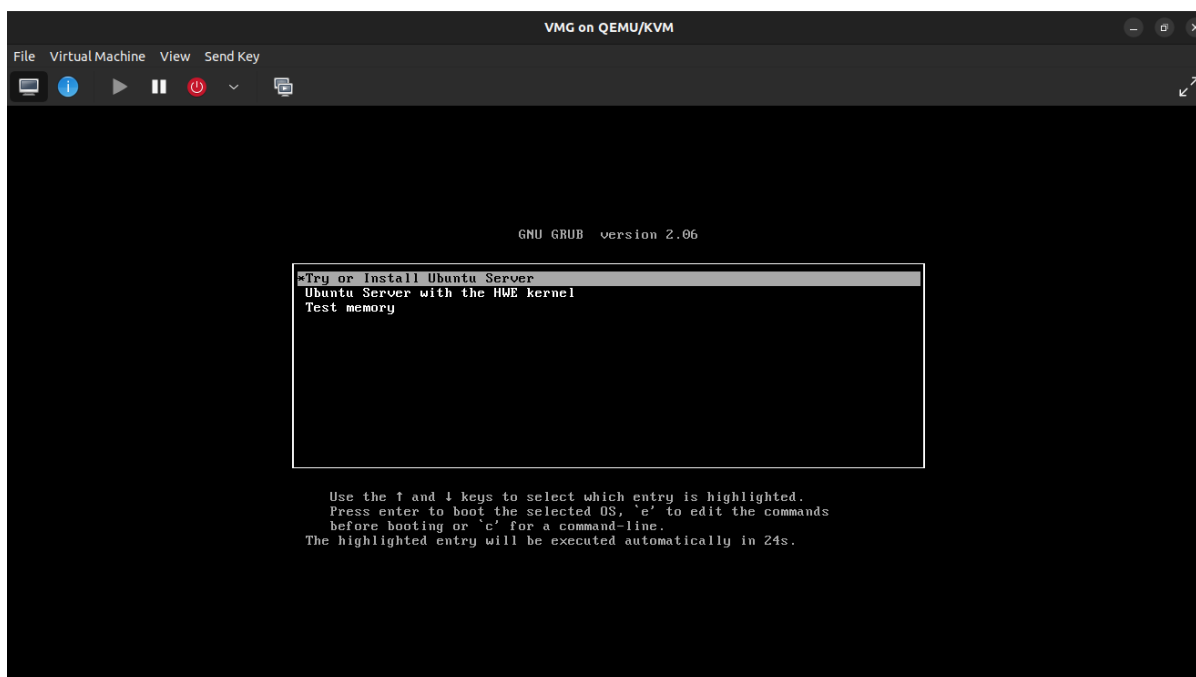
Seleccionamos la opción “Create a disk image for this virtual machine” e ingresamos la cantidad de espacio de disco que será otorgada a la VM. Damos clic en el botón “Forward”.



Ingresamos el nombre de la VM y damos clic en el botón “Finish”.



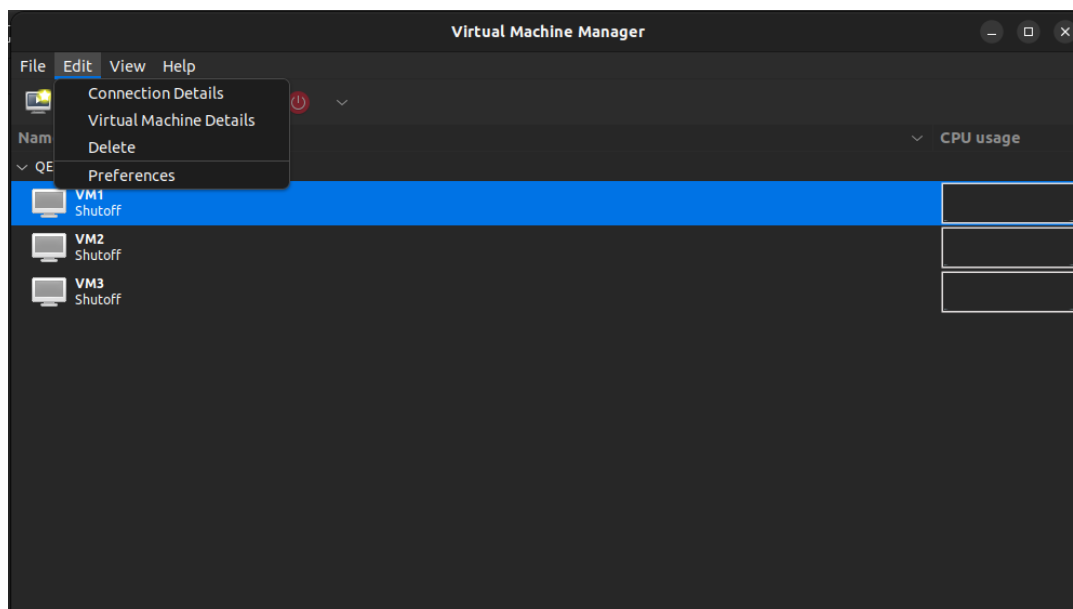
Seguimos los pasos para instalar el sistema operativo.



## Pasos para la configuración del DHCP de la red virtual

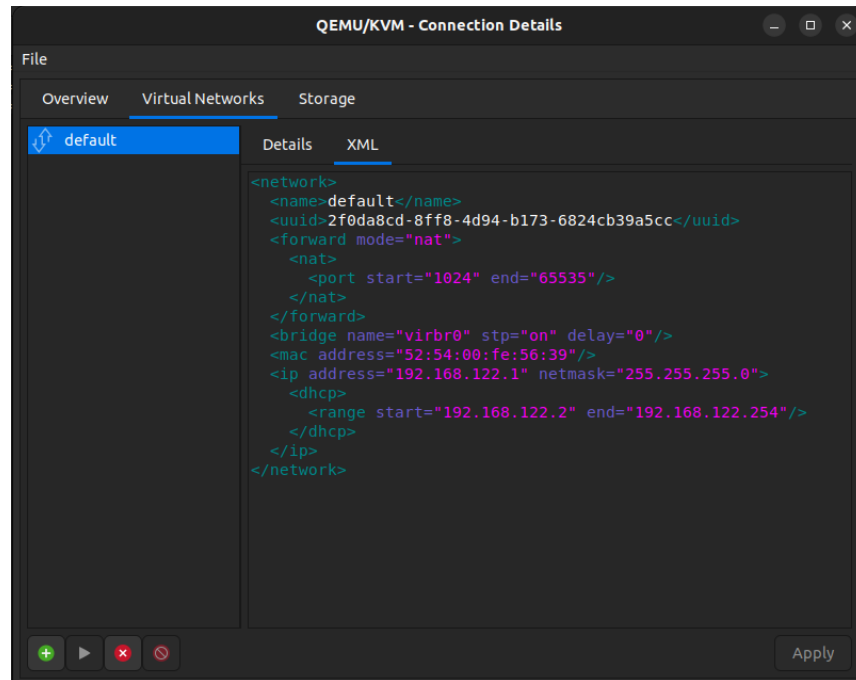
Estos pasos son necesarios para que las IP de las VM no cambien si necesitamos reiniciarlas, y así las variables de entorno no deban ser modificadas cada vez que levantemos el sistema.

Una vez creadas las 3 Vms, seleccionamos la opción “Edit” > “Connection Details”.

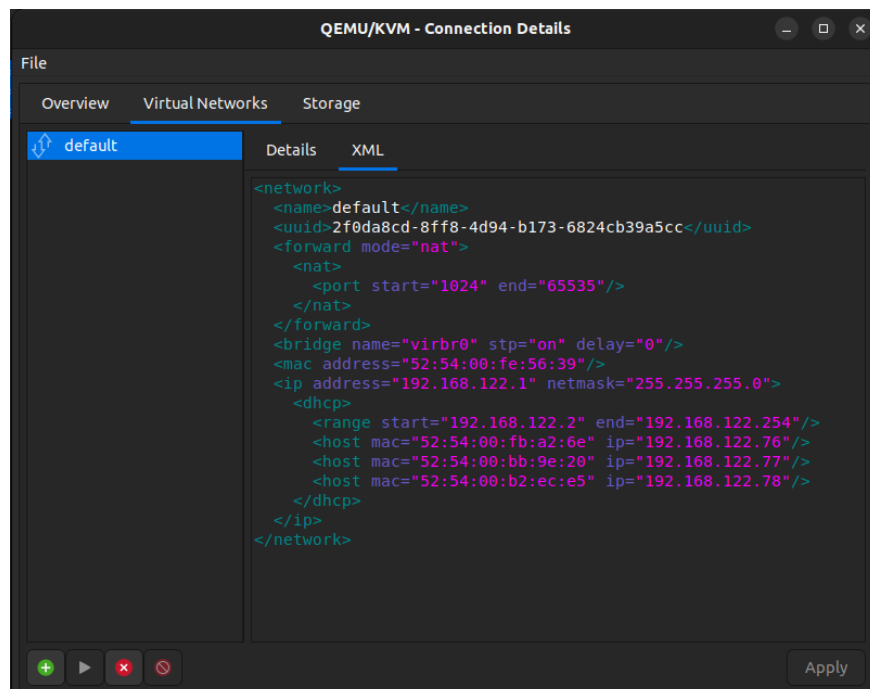




Seleccionamos la pestaña “Virtual Networks” y damos clic al botón con el círculo rojo para detener la ejecución de la red virtual.



Agregamos una etiqueta “host” por cada VM, las cuales contienen una propiedad con la dirección MAC y la IP que deseamos asignar a la VM, en la ruta “network” – “ip” – “dhcp”. Es importante que la dirección IP se encuentre dentro del rango indicado por las propiedades de la etiqueta “range” que se encuentra en la misma ruta. Al finalizar, damos clic en el botón “Apply” para guardar la configuración, y luego clic en el botón para volver a ejecutar la red virtual.



## Pasos para la configuración de la VM3

Se configurará primero la VM3, ya que será la que contenga Zot, para almacenar las imagenes de las APIs. Encendemos la VM3 e instalamos Docker usando la documentación de la página oficial de Docker ( <https://docs.docker.com/engine/install/ubuntu/> ).

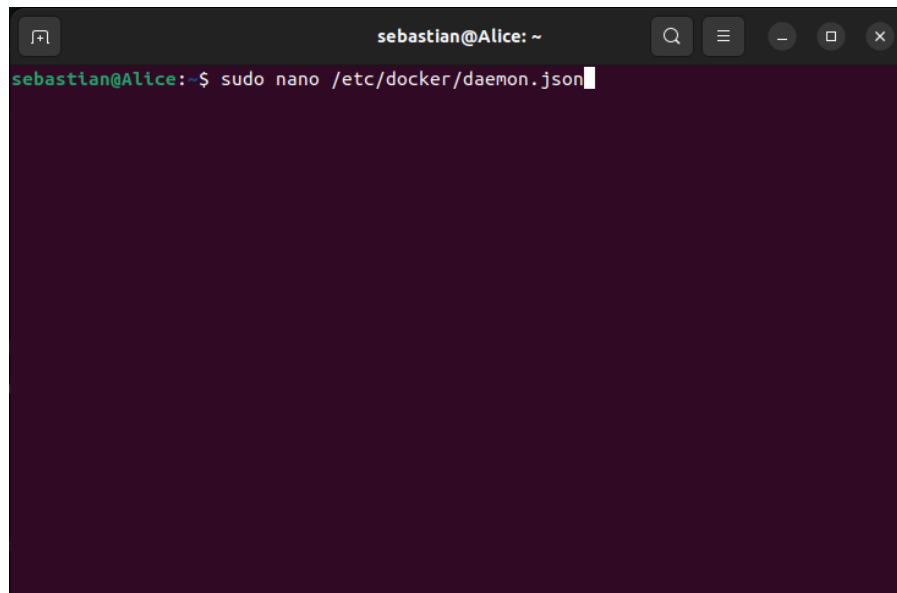
```
VM3 on QEMU/KVM
File Virtual Machine View Send Key
> Signed-By: /etc/apt/keyrings/docker.asc
> EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: jammy
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
vm3@vm3:~$ sudo apt update
Get:1 https://download.docker.com/linux/ubuntu jammy InRelease [48.5 kB]
Hit:2 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:3 http://gt.archive.ubuntu.com/ubuntu jammy InRelease
Hit:4 http://gt.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:5 http://gt.archive.ubuntu.com/ubuntu jammy-backports InRelease
Get:6 https://download.docker.com/linux/ubuntu jammy/stable amd64 Packages [68.8 kB]
Fetched 117 kB in 1s (110 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
118 packages can be upgraded. Run 'apt list --upgradable' to see them.
vm3@vm3:~$ sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  docker-ce-rootless-extras git git-man iptables less libberror-perl libip6tc2 libjansson4 libnetfilter-contrack3
  libnftnl1 libnftables1 libnftnl11 libslirp0 nftables patch pigz slirp4netns
Suggested packages:
  cgroupfs-mount | cgroup-lite docker-model-plugin git-daemon-run | git-daemon-sysvinit git-doc git-email git-gui gitk gitweb
  git-cvs git-mediawiki git-svn firewallld ed diffutils-doc
The following NEW packages will be installed:
  containerd.io docker-buildx-plugin docker-ce docker-ce-cli docker-ce-rootless-extras docker-compose-plugin git git-man
  iptables less libberror-perl libip6tc2 libjansson4 libnetfilter-contrack3 libnftnl1 libnftables1 libnftnl11 libslirp0
  nftables patch pigz slirp4netns
0 upgraded, 22 newly installed, 0 to remove and 118 not upgraded.
Need to get 102 MB of archives.
After this operation, 418 MB of additional disk space will be used.
Do you want to continue? [Y/n] _
```

Procedemos a descargar y crear un contenedor de zot e iniciarlo con el comando “docker run -d -p 5000:5000 --name zot ghcr.io/project-zot/zot:latest”.

```
VM3 on QEMU/KVM
File Virtual Machine View Send Key
vm3@vm3:~$ docker run -d -p 5000:5000 --name zot ghcr.io/project-zot/zot:latest
Unable to find image 'ghcr.io/project-zot/zot:latest' locally
latest: Pulling from project-zot/zot
46c29736e2de: Pull complete
fab8c4b3fa32: Pull complete
bfb59b82a9b6: Pull complete
017886f7e176: Pull complete
62de241dac5f: Pull complete
2780920e5dbf: Pull complete
7c12895b777b: Pull complete
3214acf845c0: Pull complete
52630fc75a18: Pull complete
dd64bf2dd177: Pull complete
4aa0ea1413d3: Pull complete
dcaa5a89b0cc: Pull complete
069d1e267530: Pull complete
e7fa9df358f0: Pull complete
4082635fdb72: Pull complete
134d6fe49b02: Pull complete
230bf33e6ef0: Pull complete
Digest: sha256:4183bd731bcc515dcf12f545d18245cfa568d338a986e4b92b7a9d89d2cc5463
Status: Downloaded newer image for ghcr.io/project-zot/zot:latest
fbb6411dcab8ace9e0217e0806fae0bc3bd813bc89caf1bfe043697af3e5e0f8
vm3@vm3:~$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
fbb6411dcab8   ghcr.io/project-zot/zot:latest      "/usr/local/bin/zot-..." 2 minutes ago  Up 2 minutes  0.0.0.0:5000->5000/tcp,
[::]:5000->5000/tcp
vm3@vm3:~$ _
```

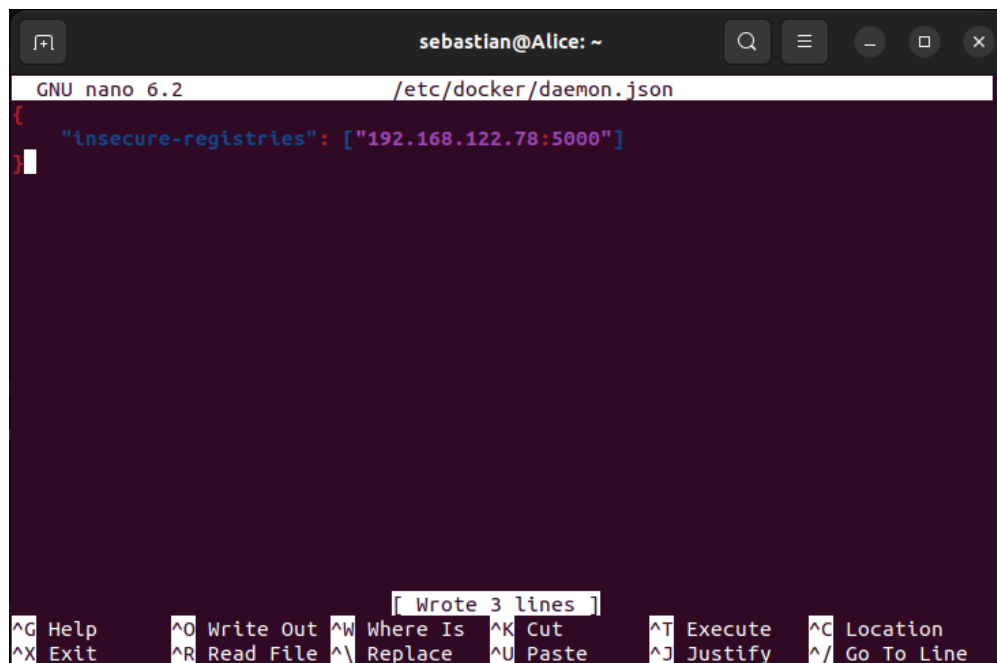
## Pasos para subir una imagen a Zot en la VM3

Abrimos el archivo de configuración de Docker en nuestro host.



```
sebastian@Alice: ~  
sebastian@Alice:~$ sudo nano /etc/docker/daemon.json
```

Agregamos la propiedad de registros inseguros, para permitir que Docker almacene nuestras imágenes en Zot en la VM3. Para esto, ingresamos la IP de la VM3, con el puerto 5000 que es donde se ejecuta nuestro contenedor de Zot en la VM. Guardamos el contenido del archivo con Ctrl+O y luego cerramos el archivo con Ctrl+X.



```
GNU nano 6.2 /etc/docker/daemon.json  
{  
  "insecure-registries": ["192.168.122.78:5000"]  
}
```

[ Wrote 3 lines ]

|         |              |             |          |            |               |
|---------|--------------|-------------|----------|------------|---------------|
| ^G Help | ^O Write Out | ^W Where Is | ^K Cut   | ^T Execute | ^C Location   |
| ^X Exit | ^R Read File | ^_ Replace  | ^U Paste | ^J Justify | ^_ Go To Line |

Procedemos a reiniciar el servicio de Docker para cargar la nueva configuración.

```
sebastian@Alice: ~  
sebastian@Alice:~$ sudo systemctl restart docker  
sebastian@Alice:~$
```

Procedemos a crear la imagen a subir. Se muestra el procedimiento para la API1, se debe replicar con las otras APIs. Ejecutamos el comando para construir la imagen.

```
sebastian@Alice: ~/Desktop/Sopes 1/Proyecto 1/201602929_LAB_SO1_152026/API1  
sebastian@Alice:~/Desktop/Sopes 1/Proyecto 1/201602929_LAB_SO1_152026/API1$ docker build -t api1-201602929:v1 .  
[+] Building 6.7s (9/9) FINISHED  
=> [internal] load build definition from Dockerfile  
=> => transferring Dockerfile: 570B  
=> [internal] load metadata for docker.io/library/golang:1.23.6-alpine  
=> [internal] load .dockerignore  
=> => transferring context: 2B  
=> [1/4] FROM docker.io/library/golang:1.23.6-alpine@sha256:f8113c4b13e2a8b3a168dceae88ac27743cc84e959f43b9dbd2291e9c3f57a0  
=> => resolve docker.io/library/golang:1.23.6-alpine@sha256:f8113c4b13e2a8b3a168dceae88ac27743cc84e959f43b9dbd2291e9c3f57a0  
=> [internal] load build context  
=> => transferring context: 84B  
=> CACHED [2/4] WORKDIR /app  
=> CACHED [3/4] COPY . .  
=> CACHED [4/4] RUN CGO_ENABLED=0 GOOS=linux go build -o api1 .  
=> exporting to image  
=> => exporting layers  
=> => exporting manifest sha256:8356e7482790a231f7f8d356c797e2a77038402e861b53b0f9c515ab1cbb036c  
=> => exporting config sha256:a941305bbcae0d5b5c7c860d7b78e1fbb007aba7da01c63d73966394b449e1c  
=> => exporting attestation manifest sha256:e6f35b67a63e81390e473399f5d7e42f8fe6c852a2619a046309c961bd752d1  
=> => exporting manifest list sha256:230810f52f464e3d500a62d9e8884ff2aad7dce4902ce031ac38ec933dd4b40  
=> => naming to docker.io/library/api1-201602929:v1  
=> => unpacking to docker.io/library/api1-201602929:v1  
sebastian@Alice:~/Desktop/Sopes 1/Proyecto 1/201602929_LAB_SO1_152026/API1$
```

Etiquetamos la imagen para que su almacen remoto sea nuestro contenedor de Zot.

```
sebastian@Alice: ~/Desktop/Sopes 1/Proyecto 1/201602929_LAB_SO1_152026/API1
[+] Building 6.7s (9/9) FINISHED
=> [internal] load build definition from dockerfile
=> => transferring dockerfile: 570B
=> [internal] load metadata for docker.io/library/golang:1.23.6-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/4] FROM docker.io/library/golang:1.23.6-alpine@sha256:f8113c4b13e2a8b3a168dceaae88ac27743cc84e959f43b9dbd2291e9c3f57a0
=> => resolve docker.io/library/golang:1.23.6-alpine@sha256:f8113c4b13e2a8b3a168dceaae88ac27743cc84e959f43b9dbd2291e9c3f57a0
=> [internal] load build context
=> => transferring context: 84B
=> CACHED [2/4] WORKDIR /app
=> CACHED [3/4] COPY . .
=> CACHED [4/4] RUN CGO_ENABLED=0 GOOS=linux go build -o api1 .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:8356e7482790a231f7f8d356c797e2a77038402e861b53b0f9c515ab1cbb036c
=> => exporting config sha256:a941305bbcaee0d5b5c7c860d7b78e1fbb007aba7da01c63d73966394b449e1c
=> => exporting attestation manifest sha256:e6f35b67a63e81390e473399f5d7e42f8fe6c852a26194a046309c961bd752d1
=> => exporting manifest list sha256:230810f52f464e3d5000a62d9e8884ff2aad7dce4902ce031ac38ec933dd4b40
=> => naming to docker.io/library/api1-201602929:v1
=> => unpacking to docker.io/library/api1-201602929:v1
sebastian@Alice:~/Desktop/Sopes 1/Proyecto 1/201602929_LAB_SO1_152026/API1$ docker tag api1-201602929:v1 192.168.122.78:5000/api1-201602929:v1
sebastian@Alice:~/Desktop/Sopes 1/Proyecto 1/201602929_LAB_SO1_152026/API1$
```

Subimos la imagen a nuestro contenedor de Zot en la VM3.

```
sebastian@Alice: ~/Desktop/Sopes 1/Proyecto 1/201602929_LAB_SO1_152026/API1
[+] Building 6.7s (9/9) FINISHED
=> [internal] load build definition from dockerfile
=> => transferring dockerfile: 570B
=> [internal] load metadata for docker.io/library/golang:1.23.6-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/4] FROM docker.io/library/golang:1.23.6-alpine@sha256:f8113c4b13e2a8b3a168dceaae88ac27743cc84e959f43b9dbd2291e9c3f57a0
=> => resolve docker.io/library/golang:1.23.6-alpine@sha256:f8113c4b13e2a8b3a168dceaae88ac27743cc84e959f43b9dbd2291e9c3f57a0
=> [internal] load build context
=> => transferring context: 84B
=> CACHED [2/4] WORKDIR /app
=> CACHED [3/4] COPY . .
=> CACHED [4/4] RUN CGO_ENABLED=0 GOOS=linux go build -o api1 .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:8356e7482790a231f7f8d356c797e2a77038402e861b53b0f9c515ab1cbb036c
=> => exporting config sha256:a941305bbcaee0d5b5c7c860d7b78e1fbb007aba7da01c63d73966394b449e1c
=> => exporting attestation manifest sha256:e6f35b67a63e81390e473399f5d7e42f8fe6c852a26194a046309c961bd752d1
=> => exporting manifest list sha256:230810f52f464e3d5000a62d9e8884ff2aad7dce4902ce031ac38ec933dd4b40
=> => naming to docker.io/library/api1-201602929:v1
=> => unpacking to docker.io/library/api1-201602929:v1
sebastian@Alice:~/Desktop/Sopes 1/Proyecto 1/201602929_LAB_SO1_152026/API1$ docker tag api1-201602929:v1 192.168.122.78:5000/api1-201602929:v1
sebastian@Alice:~/Desktop/Sopes 1/Proyecto 1/201602929_LAB_SO1_152026/API1$ docker push 192.168.122.78:5000/api1-201602929:v1
The push refers to repository [192.168.122.78:5000/api1-201602929]
720ae4c6db70: Pushed
2dab78eab6f2: Pushed
dd968b00abc2: Pushed
f18232174bc9: Pushed
4f4fb700ef54: Pushed
5a4100860a27: Pushed
1cfa5db9ef3b: Pushed
05bcb020fb98: Pushed
824ef5f9d3cd: Pushed
v1: digest: sha256:230810f52f464e3d5000a62d9e8884ff2aad7dce4902ce031ac38ec933dd4b40 size: 856
sebastian@Alice:~/Desktop/Sopes 1/Proyecto 1/201602929_LAB_SO1_152026/API1$
```

## Pasos para la configuración de la VM1

Instalamos Containerd. Utilizamos la siguiente secuencia de comandos:

- > sudo apt update
- > sudo apt install -y ca-certificates curl software-properties-common
- > sudo apt install -y containerd
- > sudo mkdir -p /etc/containerd
- > sudo containerd config default | sudo tee /etc/containerd/config.toml
- > sudo systemctl restart containerd
- > sudo systemctl enable containerd

```
File Virtual Machine View Send Key
vm1@vm1:~$ sudo apt install -y containerd
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  runc
The following NEW packages will be installed:
  containerd runc
0 upgraded, 2 newly installed, 0 to remove and 83 not upgraded.
Need to get 47.4 MB of archives.
After this operation, 178 MB of additional disk space will be used.
Get:1: http://gt.archive.ubuntu.com/ubuntu jammy-updates/main amd64 runc amd64 1.3.3-0ubuntu1~22.04.3 [8857 kB]
Get:2: http://gt.archive.ubuntu.com/ubuntu jammy-updates/main amd64 containerd amd64 1.7.28-0ubuntu1~22.04.1 [38.5 MB]
Fetched 47.4 MB in 13s (3712 kB/s)
debconf: delaying package configuration, since apt-utils is not installed
Selecting previously unselected package runc.
(Reading database ... 62093 files and directories currently installed.)
Preparing to unpack .../runc_1.3.3-0ubuntu1~22.04.3_amd64.deb ...
Unpacking runc (1.3.3-0ubuntu1~22.04.3) ...
Selecting previously unselected package containerd.
Preparing to unpack .../containerd_1.7.28-0ubuntu1~22.04.1_amd64.deb ...
Unpacking containerd (1.7.28-0ubuntu1~22.04.1) ...
Setting up runc (1.3.3-0ubuntu1~22.04.3) ...
Setting up containerd (1.7.28-0ubuntu1~22.04.1) ...
Created symlink /etc/systemd/system/multi-user.target.wants/containerd.service → /lib/systemd/system/containerd.service.
/usr/sbin/policy-rc.d returned 101, not running 'start containerd.service'
debconf: unable to initialize frontend: Dialog
debconf: (No usable dialog-like program is installed, so the dialog based frontend cannot be used. at /usr/share/perl5/Debconf/FrontEnd/Dialog.pm line 78.)
debconf: falling back to frontend: Readline
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.
```

Procedemos a descargar la imagen de la API1.

```
File Virtual Machine View Send Key
vm1@vm1:~$ sudo ctr images pull --plain-http 192.168.122.78:5000/apl1-201602929:v1
192.168.122.78:5000/apl1-201602929:v1: resolved |+++++|
+++++|
Index-sha256:230810f52f464e3d5000a62d9e8884ff2aad7dce4902ce031ac38ec933dd4b40: done |+++++|
+++++|
manifest-sha256:8356e7482790a231f7f8d356c797e2a77038402e061b53b0f9c515ab1cbb036c: done |+++++|
+++++|
layer-sha256:1cfa5db9ef3b4484dd9aaadbf66392d9f197db0fba63219d4efae9f6a987f904: done |+++++|
+++++|
config-sha256:a941305bbcaee0d5b5c7c860d7b78e1fbb007aba7da01c63d73966394b449e1c: done |+++++|
+++++|
layer-sha256:f18232174bc91741fd93da96d85011092101a032a93a388b79e99e69c2d5c870: done |+++++|
+++++|
layer-sha256:2dab7eab6f2d725c7de864cc217a5c504a2dbd61a5e426545eca979fc42df3c: done |+++++|
+++++|
layer-sha256:dd968b00abc2f44f5d74b014d2b833a05dd020b5f34f19dca853c409df33466: done |+++++|
+++++|
layer-sha256:05cb020fb9889a3721af34f0eaa3f0ee3b319cc797aa66b03b5632603cb66a: done |+++++|
+++++|
layer-sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0c0b5577484a6d75e68dc38e8acc1: done |+++++|
+++++|
layer-sha256:824ef5f9d3cdaab1e16029ec426eae55022c77632c9df1e7c5d28253e56761a9: done |+++++|
+++++|
layer-sha256:5a4100860a270959a581af61fafcd4c22ad923f4360730e0e8f4f8722e30db8f: done |+++++|
+++++|
elapsed: 5.6 s total: 92.5 M (16.4 MiB/s)

unpacking linux/amd64 sha256:230810f52f464e3d5000a62d9e8884ff2aad7dce4902ce031ac38ec933dd4b40...
done: 14.916882241s
vm1@vm1:~$ _
```

Descargamos la imagen de la API2.

```
Virtual Machine  View  Send Key
[info] [play] [pause] [stop] [power] [fullscreen] [copy]

vm1@vm1:~$ sudo ctr images pull --plain-http 192.168.122.78:5000/api2-201602929:v1
192.168.122.78:5000/api2-201602929:v1: resolved |+++++
+++++|
index-sha256:d213cbb902c43d42bf84da3afe4913eadfba0c1dd584114a067740831b7e8d89: done |+++++
+++++|
manifest-sha256:e1ff811ae7bd43c901f07713394d693171a0ce7f998a8ea4bf7fb7857377ffe3: done |+++++
+++++|
layer-sha256:5c06bc7b50274239959c90db102ee9a160d49a00ceae3cdfea124cc1acd18171: done |+++++
+++++|
config-sha256:f02583d0a46357f065d109da169ba18ab1bebab4b6b1e7ee750efa621057ce01: done |+++++
+++++|
layer-sha256:f18232174bc91741fdf3da96d85011092101a032a93a388b79e99e69c2d5c870: exists |+++++
+++++|
layer-sha256:2dab78eab6f2d725c7de864cc217a5c504a2dbd61a5e426545eca979fc42df3c: exists |+++++
+++++|
layer-sha256:dd968b00abc2f44f5d74b014d2b833a05dd020b5f34f19dca853c409df33466: exists |+++++
+++++|
layer-sha256:05bcb020fb9889a3721af34f0eaa3f0fee3b319cc797aa66b03b5632603cb66a: exists |+++++
+++++|
layer-sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1: exists |+++++
+++++|
layer-sha256:824ef5f9d3cdaab1e16029ec426eae55022c77632c9df1e7c5d28253e56761a9: exists |+++++
+++++|
layer-sha256:f38cb455cd9f19aa053fd105ceb8af35826e7680badd57cb86319665d00c98fe: done |+++++
+++++|
elapsed: 4.2 s total: 21.6 M (5.1 MiB/s)

unpacking linux/amd64 sha256:d213cbb902c43d42bf84da3afe4913eadfba0c1dd584114a067740831b7e8d89...
done: 5.307497449s
vm1@vm1:~$ _
```

Creamos y ejecutamos en segundo plano un contenedor para la API1. También creamos y ejecutamos en segundo plano un contenedor para la API2. Pasamos en estos comandos la variable de entorno de la URL de la VM2.

```
File  Virtual Machine  View  Send Key
[monitor] [info] [play] [pause] [stop] [power] [fullscreen] [copy]

vm1@vm1:~$ sudo ctr run -d --env VM2_URL=192.168.122.77 --net-host 192.168.122.78:5000/api1-201602929:v1 api1
Iniciando API1 en el puerto : 8081
vm1@vm1:~$ sudo ctr run -d --env VM2_URL=192.168.122.77 --net-host 192.168.122.78:5000/api2-201602929:v1 api2
Iniciando API2 en el puerto : 8082
vm1@vm1:~$ _
```

## Pasos para la configuración de la VM2

Instalamos Containerd. Utilizamos la siguiente secuencia de comandos:

- > sudo apt update
- > sudo apt install -y ca-certificates curl software-properties-common
- > sudo apt install -y containerd
- > sudo mkdir -p /etc/containerd
- > sudo containerd config default | sudo tee /etc/containerd/config.toml
- > sudo systemctl restart containerd
- > sudo systemctl enable containerd

```
File Virtual Machine View Send Key
vm2@vm2:~$ sudo apt install -y containerd
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  runc
The following NEW packages will be installed:
  containerd runc
0 upgraded, 2 newly installed, 0 to remove and 118 not upgraded.
Need to get 47.4 MB of archives.
After this operation, 178 MB of additional disk space will be used.
Get:1 http://gt.archive.ubuntu.com/ubuntu jammy-updates/main amd64 runc amd64 1.3.3-0ubuntu1~22.04.3 [8657 kB]
Get:2 http://gt.archive.ubuntu.com/ubuntu jammy-updates/main amd64 containerd amd64 1.7.28-0ubuntu1~22.04.1 [38.5 MB]
Fetched 47.4 MB in 11s (4152 kB/s)
debconf: delaying package configuration, since apt-utils is not installed
Selecting previously unselected package runc.
(Reading database ... 62093 files and directories currently installed.)
Preparing to unpack .../runc_1.3.3-0ubuntu1~22.04.3_amd64.deb ...
Unpacking runc (1.3.3-0ubuntu1~22.04.3) ...
Selecting previously unselected package containerd.
Preparing to unpack .../containerd_1.7.28-0ubuntu1~22.04.1_amd64.deb ...
Unpacking containerd (1.7.28-0ubuntu1~22.04.1) ...
Setting up runc (1.3.3-0ubuntu1~22.04.3) ...
Setting up containerd (1.7.28-0ubuntu1~22.04.1) ...
Created symlink /etc/systemd/system/multi-user.target.wants/containerd.service → /lib/systemd/system/containerd.service.
/usr/sbin/policy-rc.d returned 101, not running 'start containerd.service'
debconf: unable to initialize frontend: Dialog
debconf: (No usable dialog-like program is installed, so the dialog based frontend cannot be used. at /usr/share/perl5/Debconf/FrontEnd/Dialog.pm line 78.)
debconf: falling back to frontend: Readline
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.
```

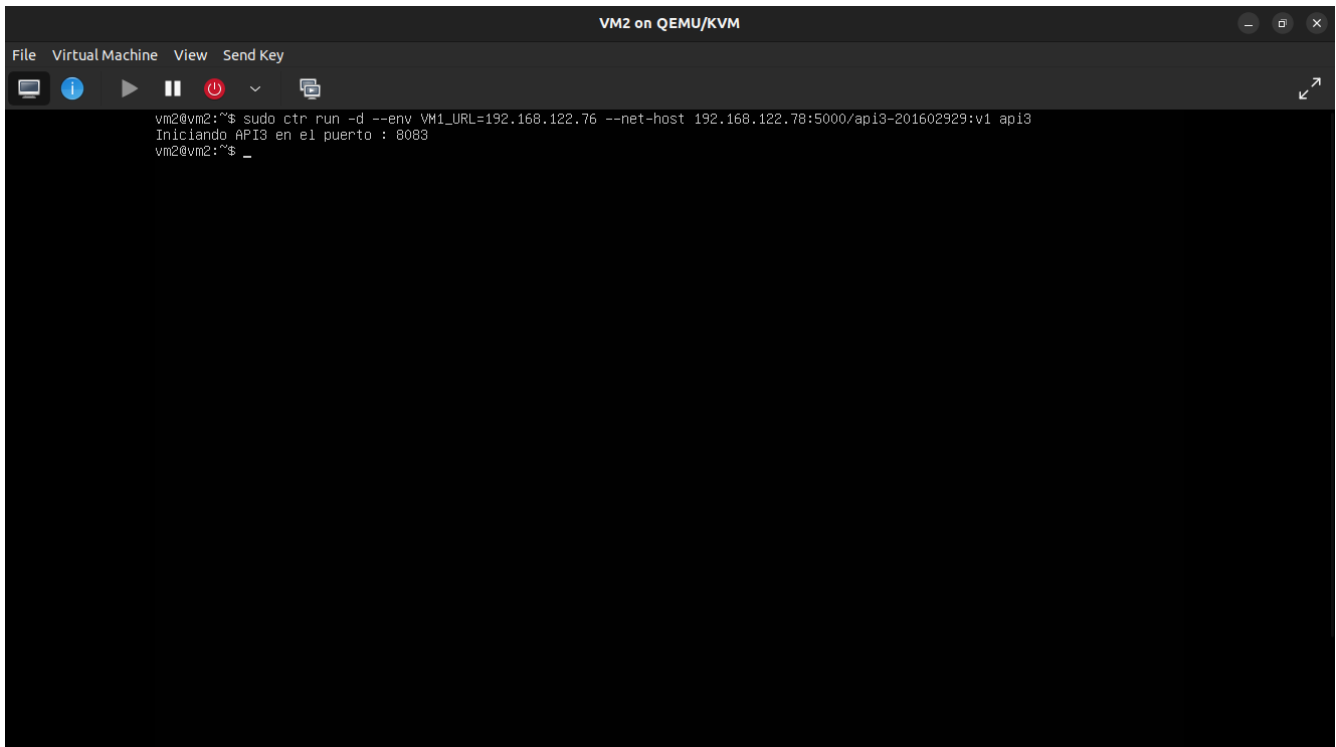
Descargamos la imagen de la API3.

```
File Virtual Machine View Send Key
vm2@vm2:~$ sudo ctr images pull --plain-http 192.168.122.78:5000/api3-201602929:v1
[sudo] password for vm2:
192.168.122.78:5000/api3-201602929:v1 resolved |+++++|
+++++|
index-sha256:b8288ff7fa805c0803a6527df76e65eb3dd72e7f3a842eb49c80ce3fcae9b008: done |+++++|
+++++|
manifest-sha256:6fe96679002a4cf214bac9d479608dd4001d90f759344b18a19a9947c9053588: done |+++++|
+++++|
layer-sha256:8b78c163f0fd6492bd7b00e7309d9bba324c1bb2a469f8438bfa53dc8fd4d917: done |+++++|
+++++|
config-sha256:ccf84bf69bb11775049d58136c00fbab0a7a672a8ef97e9c9ba83777bd0b196f: done |+++++|
+++++|
layer-sha256:f18232174bc91741fd93da96d85011092101a032a93a388b79e99e69c2d5c870: done |+++++|
+++++|
layer-sha256:2dab78eab6f2d725c7de864cc217a5c504a2dbd61a5e426545eca979fc42df3c: done |+++++|
+++++|
layer-sha256:dd966b00abc2f44f5d74b014d2b833a05dd020b5f534f19dca853c409df33466: done |+++++|
+++++|
layer-sha256:05bcb020fb9889a3721af34f0eaa3f0fee3b319cc797aa66b03b5632603cb66a: done |+++++|
+++++|
layer-sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0c0b5577484a6d75e68dc38e8acc1: done |+++++|
+++++|
layer-sha256:824ef5f9d3cdaab1e16029ec426eae55022c77632c9df1e7c5d28253e56761a9: done |+++++|
+++++|
layer-sha256:81b8cafa9820979f0a5edac23703695ac0ab529fdb5d91af1045c25b1b0c8104f: done |+++++|
+++++|
elapsed: 9.9 s total: 70.6 M (7.1 MiB/s)

unpacking linux/amd64 sha256:b8288ff7fa805c0803a6527df76e65eb3dd72e7f3a842eb49c80ce3fcae9b008...
done: 14.08272677s
vm2@vm2:~$ _
```



Creamos y ejecutamos en segundo plano un contenedor para la API3. Enviamos la variable de entorno para la URL de la VM1 en el comando.



```
VM2 on QEMU/KVM
File Virtual Machine View Send Key
vm2@vm2:~$ sudo ctr run -d --env VM1_URL=192.168.122.76 --net-host 192.168.122.78:5000/ap13-201602929:v1 ap13
Iniciando API3 en el puerto : 8083
vm2@vm2:~$ _
```